Spatial Data Management / Multidimensional Databases & Data Warehousing / Delta Lake

PART A

1. Which option best describes a **topological** relationship between two spatial objects?

A. “North of”

B. “Within 200 m”

C. “Intersects”

D. “Closer to A than to B”

2. In the **filter–refine** framework, the **filter** step typically uses:

A. Exact polygon intersection tests

B. Minimum Bounding Rectangles (MBRs) to generate candidates

C. A full scan of all object vertices

D. Only directional predicates (e.g., left/right)

3. A primary reason spatial query processing is challenging is that multidimensional space:

A. Has a unique total ordering that preserves proximity

B. Has no total ordering that preserves spatial proximity well

C. Always supports O(1) nearest-neighbour search

D. Eliminates the need for indexing

4. In an R-tree range query, when should the search descend into a child entry?

A. Only if the child object is guaranteed to satisfy the query

B. Only if the child MBR is fully contained in the query window

C. If the child MBR intersects the query window

D. Only if the child MBR has minimum area among siblings

5. In R-trees, **dead space** refers to:

A. Space inside an MBR that contains no qualifying object geometry

B. Empty pages that cannot be allocated

C. Space outside the query window

D. Duplicate object IDs caused by hashing

6. Which issue most directly causes an R-tree to visit multiple paths for the same query window?

A. Perfect partitioning with no overlap

B. Overlap between directory MBRs

C. Use of exact geometry in directory entries

D. Sorting objects by a 1D key

7. A key limitation of **space-filling curves** (e.g., Z-order, Hilbert) is that:

A. They cannot map 2D space to 1D keys

B. They guarantee perfect proximity preservation for all datasets

C. They map 2D regions to intervals on a 1D ordering, but proximity is not always preserved

D. They only work for triangles, not rectangles

8. In a multidimensional model, a **fact** most directly represents:

A. A hierarchy level (e.g., Year $\rightarrow$ Month $\rightarrow$ Day)

B. A business event/phenomenon with associated measures (e.g., a sale)

C. A textual description of a product

D. A physical storage structure for indexing

9. A **dimension** is best described as:

A. A numeric measure that can always be summed

B. Context describing facts (often organised into hierarchies)

C. A table containing only timestamps

D. A table that stores only aggregated values

10. A **star schema** typically consists of:

A. A central fact table connected to denormalised dimension tables

B. Only dimension tables, no facts

C. Fully normalised dimensions split into many sub-tables

D. One table per OLAP query

11. Compared to a star schema, a **snowflake schema** mainly:

A. Removes the fact table

B. Denormalises dimensions for faster joins

C. Normalises dimensions into multiple related tables

D. Stores only non-additive measures

12. Which measure type can be meaningfully aggregated **along all dimensions**?

A. Additive

B. Semi-additive

C. Non-additive

D. Directional

13. A typical **semi-additive** measure is:

A. Total sales revenue

B. Inventory level

C. Average rating (computed)

D. A product identifier

14. Which OLAP operation moves from a more detailed level to a more general level in a hierarchy?

A. Drill-down

B. Roll-up

C. Dice

D. Slice

15. Which statement best describes **slice** vs **dice**?

A. Slice selects a sub-cube by fixing one dimension to a single value; dice selects a sub-cube by constraining ranges/sets on multiple dimensions

B. Slice always increases dimensionality; dice always decreases it

C. Slice is only for measures; dice is only for dimensions

D. They are synonyms

16. OLAP differs from OLTP primarily because OLAP:

A. Focuses on short, highly concurrent update transactions

B. Focuses on analytical queries over integrated, historical data with a multidimensional view

C. Avoids aggregations to preserve raw detail

D. Requires strict row-by-row procedural logic

17. A key challenge in using **cloud object stores** as table storage is that they typically:

A. Provide free and instantaneous directory renames

B. Provide strong cross-key consistency by default for all operations

C. Make metadata operations like LIST expensive and/or eventually consistent

D. Prevent parallel reads of data objects

18. In Delta Lake, the transaction log is stored:

A. Inside each Parquet file footer only

B. In a special subdirectory (e.g., `_delta_log`) as versioned log objects

C. Only in an external database service

D. Only in memory during query execution

19. Delta Lake supports **snapshot reads** and **serializable writes** meaning that:

A. Readers always see partially committed changes

B. Writers never conflict because conflicts are ignored

C. Readers see a consistent table version; writers commit changes in a serializable order

D. Reads are serializable but writes are not

20. The main purpose of **checkpoints** in Delta Lake is to:

A. Replace Parquet with JSON for data storage

B. Summarise log history to speed up table state reconstruction

C. Disable time travel to save space

D. Force all files into a single partition

PART B

1. Explain the difference between the topological relationships **disjoint**, **intersects**, and **equals** using the concepts of **interior** and **boundary** of spatial objects. Give a short example for each (you may describe geometries in words).

2. Consider a spatial selection query "Find all objects that intersect a given query window". Explain **step-by-step** how the **filter–refine** framework evaluates this query using **MBRs**, and explain why the filter step can produce **false positives**. Illustrate with a small concrete example (described in words or ASCII).

3. You are designing a data warehouse for book sales. Propose a **star schema** by specifying:
• the **fact table** (including its grain) and at least two **measures**,
• at least three **dimensions** (with an example of a hierarchy for at least one dimension), and
• for each measure, classify it as **additive**, **semi-additive**, or **non-additive**, with justification.

4. Consider a sales cube with dimensions **Time** (Day → Month → Year), **Location** (Store → City → Country), and **Product** (Item → Category), and measure **Sales**. Describe an analysis workflow using a sequence of OLAP operations that includes **roll-up**, **drill-down**, **slice**, and **dice**. For each step, state what changes in the resulting cube.

5. Describe the **Delta Lake write protocol** at a high level, explaining how the transaction log enables **serializable writes** on top of a cloud object store. Your answer should cover:
• how writers choose the next table version,
• writing data objects vs writing the log record,
• what it means for the transaction to "commit atomically", and
• the role of **checkpoints** and the `_last_checkpoint` pointer for read performance.

Solutions

PART A — Answer Key + Justifications

1: C — “Intersects” is a topological predicate (based on how geometries overlap), unlike directional or distance-based predicates.

2: B — The filter step uses cheap approximations (MBRs) to prune and produce a candidate set; exact geometry checks are deferred.

3: B — Multidimensional data lacks a total order that preserves proximity, complicating indexing and cost guarantees.

4: C — R-tree range queries descend into entries whose MBR intersects the query window (necessary condition, not sufficient).

5: A — Dead space is area covered by an MBR that does not actually contain relevant object geometry, causing false positives.

6: B — Overlap between directory MBRs forces exploring multiple subtrees because both may intersect the query window.

7: C — Space-filling curves map multidimensional space to a 1D order; locality is improved but not perfectly preserved.

8: B — Facts represent business events/phenomena (e.g., sales) and contain measures plus links to dimensions.

9: B — Dimensions provide context for facts and are typically organised into hierarchies for roll-up/drill-down.

10: A — Star schemas have a central fact table with denormalised dimensions around it.

11: C — Snowflake schemas normalise dimensions into multiple tables (more joins, more explicit structure).

12: A — Additive measures can be aggregated along all dimensions (e.g., summing sales).

13: B — Inventory level is classic semi-additive: summable across products/locations, but not meaningfully across time snapshots.

14: B — Roll-up aggregates to a higher (more general) level (e.g., Day $\rightarrow$ Month $\rightarrow$ Year).

15: A — Slice fixes a single value of one dimension; dice constrains multiple dimensions (ranges/sets), yielding a sub-cube.

16: B — OLAP supports analytical, often aggregate-heavy queries over historical/integrated data; OLTP is short update-centric transactions.

17: C — Object stores make metadata operations (e.g., LIST) slow/complex and may have eventual consistency behaviours.

18: B — Delta tables keep the log in a dedicated log area (commonly _delta_log) with versioned entries.

19: C — Snapshot reads mean a reader sees a consistent version; serializable writes mean committed updates appear in a serial order.

20: B — Checkpoints summarise log history to avoid replaying many log records when reconstructing table state.

PART B — Marking Guides + Model Answers

B1) Topological relationships via interior/boundary

Key points expected

- Define interior(o) and boundary(o) and use them to distinguish predicates.
- Correct conceptual difference among disjoint/intersects/equals.
- Simple examples (e.g., two polygons separate / touching / identical).

Common mistakes to avoid

- Confusing “intersects” with “overlaps only” (intersects can include boundary contact).
- Using distance/direction language instead of topology.

Model answer

Topological predicates depend on how geometries relate through their **interiors** and **boundaries**. Two objects are **disjoint(o1, o2)** if neither their interiors intersect nor their boundaries touch; intuitively, they are completely separate with a gap between them. They **intersect(o1, o2)** if either their interiors overlap or their boundaries touch (so even “just touching at the edge” counts as intersects). They are **equals(o1, o2)** if they represent the same geometry—i.e., they cover exactly the same set of points (same interior and boundary). Example: two non-touching rectangles are disjoint; a rectangle whose edge touches another rectangle’s edge intersects; two identical polygons equal.

B2) Filter–refine with MBRs (step-by-step)

Key points expected

- Clearly separate filter step (MBR test) vs refine step (exact geometry test).
- Explain candidate set + why false positives happen (MBR is approximation).
- A small worked example (described) showing at least one false positive.

Common mistakes to avoid

- Claiming MBR intersection implies true intersection of exact geometry.
- Skipping the refinement step.

Model answer

To evaluate “objects that intersect a query window”, the **filter–refine** framework uses a cheap approximation first, then exact geometry:
1) Compute/lookup each object’s **MBR** (minimum rectangle enclosing the object).
2) **Filter step:** test which MBRs intersect the query window. Any object whose MBR does **not** intersect can be safely discarded. The survivors form the **candidate set**.
3) **Refine step:** for each candidate, run the exact geometric predicate (e.g., polygon-window intersection) using the true geometry. Keep only those that truly intersect.

False positives occur because an MBR is an over-approximation: an MBR can intersect the query window even if the object geometry inside the MBR does not. Example: an object is a “C-shaped” polygon; its MBR is a full rectangle. A small query window might intersect the rectangle area that falls in the “gap” of the C-shape—so the MBR test passes, but the exact geometry test fails.

B3) Star schema design + measure types

Key points expected

- Fact table with explicit grain.
- At least 3 dimensions, with at least one hierarchy example.
- Two measures with correct additive/semi-additive/non-additive classification + justification.

Common mistakes to avoid

- Missing grain (schema becomes ambiguous).
- Calling an average “additive” without addressing why it’s non-additive.

Model answer

A suitable **star schema**:

FactSales (grain: one row per sold item per store per day):
• Foreign keys: `time_key`, `store_key`, `product_key` (optionally `customer_key`)
• Measures: `sales_amount` (additive), `units_sold` (additive), optionally `avg_discount_rate` (non-additive; requires weighting).

DimTime: Day → Month → Year (date, day_name, month, quarter, year)
DimStore/Location: Store → City → Country (store_id, city, country, region)
DimProduct: Item → Category (item, category, brand)

If inventory is tracked in a snapshot fact (e.g., store-product-day), `inventory_level` is **semi-additive**: it aggregates across products/locations but not across time without care.

B4) OLAP workflow with roll-up, drill-down, slice, dice

Key points expected

- Correct definitions of roll-up, drill-down, slice, dice.
- A coherent sequence where each operation changes granularity or subsetting.
- Explicitly state what changes in the cube after each step.

Common mistakes to avoid

- Mixing slice and dice.
- Using roll-up/drill-down backwards.

Model answer

Start with Sales by Day × Store × Item (fine granularity).
1) **Roll-up (Time):** Day → Month to view Sales by Month × Store × Item (aggregates days into months).
2) **Slice (Location):** fix Country = Portugal to restrict the cube to that single country value.
3) **Dice (Product + Time):** restrict Category ∈ {Fiction, Sci■Fi} and Year ∈ {2024, 2025} to obtain a smaller sub-cube.
4) **Drill-down (Location):** move from City/Country level to Store level to inspect which stores drive results (more detail).

B5) Delta Lake write protocol + serializable writes

Key points expected

- Writers read current version r and attempt to create the next version record r+1 in the log.
- Write data files first, then commit by writing the log record.
- Commit atomicity is defined by the successful creation of the next log record.
- Checkpoints + _last_checkpoint reduce log replay and speed reads.

Common mistakes to avoid

- Claiming the table is updated when data files are written (it’s updated when the log record commits).
- Ignoring that version creation must ensure only one writer wins a given version.

Model answer

A Delta table keeps its state via a versioned **transaction log**. A writer:
1) Reads the latest table state/version **r** (often from a checkpoint plus newer log entries).
2) Writes new data objects (e.g., Parquet files) to the object store using unique names (no in-place overwrite).
3) Attempts to write the next log record `r+1.json` that lists actions (add/remove files, metadata).
The transaction **commits atomically** when this new log record is successfully created—this defines the new snapshot version. Readers obtain **snapshot reads** by reading a consistent version; writers achieve **serializable writes** because versions are committed in a single ordered sequence and conflicts are detected. Periodic **checkpoints** summarise state up to a version, and `_last_checkpoint` points readers to the latest checkpoint to avoid replaying a long log.